A CASE STUDY REPORT ON

ANALYSIS OF CLICK ISSUE #3362: HELPFORMATTER.WRITE_USAGE BREAKS OPTIONS AT A HYPHEN

REPORT SUBMITTED TOWARDS CASE STUDY OF OPEN SOURCE TECHNOLOGIES (TE7756)

BACHELOR OF TECHNOLOGY

(Robotics and Automation)

SUBMITTED BY

Bhumika Rai – 24070127035

Sara Varma - 24070127098

Saanvi Agrahari - 24070127104

Sejal Khuntia - 24070127146



UNDER THE GUIDANCE OF

Dr. Amol Kamble

SYMBIOSIS INSTITUTE OF TECHNOLOGY PUNE - 412115

A CONSTITUENT OF SYMBIOSIS INTERNATIONAL (DEEMED UNIVERSITY)

2026-27

Index

No.	Section	What it covers
1	Introduction	Problem statement, objectives, group and work division
2	Project Overview	What Click is, its features, a working demo
3	Repository Investigation	Structure, activity, labels, issues and pull requests
4	Issue Investigation	Issue #3362: description, labels, status
5	Bug Investigation	Reproduction, root cause and a supporting experiment
6	Contribution Process	How the Pallets team expects contributions
7	Pull Request Investigation	The three pull requests linked to the issue
8	Practical Observations	Problems we faced and what we learned
9	Changes / Contribution	Our test, our experimental fix, results and GitHub work
10	Conclusion	Summary of findings
	References	Links used in this report

1. Introduction

This report documents a case study of a real open source project, carried out for the course Open Source Technologies (TE7756) at Symbiosis Institute of Technology, Pune. We chose the Python library **Click** and investigated one of its open issues, **pallets/click#3362**, by reproducing it on our own computer, finding its root cause in the source code, studying how the Pallets team expects contributions, and examining the three pull requests that tried to fix it. We also wrote our own test and tried a small experimental fix, and kept all evidence in a public GitHub repository.

1.1 Problem Statement

When a command-line program built with Click prints its `--help` output, the "Usage:" line is wrapped to fit the line width. In `HelpFormatter.write_usage`, this wrapping can break an option name in the middle, at a hyphen. For example, `--max-retry-count` is printed as `--max-` on one line and `retry-count` on the next. This makes help text confusing and can mislead users about the real option names. A line break should only happen between options, never inside one.

1.2 Objectives

1. Understand the Click project and run a working example.
2. Explore the repository: structure, documentation, issues, labels and activity.
3. Investigate issue #3362 and reproduce the bug, recording the environment, steps, expected and actual results.
4. Find the root cause in the source code and confirm it with a small experiment.
5. Study the contribution process and the pull requests linked to the issue.
6. Make our own contribution (a test and an experimental fix) and report the results honestly.

1.3 Group and Division of Work

Our group has four members. Each member took one part of the case study, and all of us worked through the whole case study so that each can explain every section.

Member	Name and PRN	Part
Student 1	Bhumika Rai - 24070127035	Project and repository (Sections 2 and 3)
Student 2	Sejal Khuntia - 24070127146	Issue and bug investigation (Sections 4 and 5)
Student 3	Sara Varma - 24070127098	Contribution process and pull requests (Sections 6 and 7)
Student 4	Saanvi Agrahari - 24070127104	Practical observations, our test and change, and GitHub work (Sections 8 and 9)

Item	Details
Project	Click (pallets/click)
Official repository	https://github.com/pallets/click
Issue investigated	https://github.com/pallets/click/issues/3362
Our case-study repository	https://github.com/bhumika08-05/click-case-study-CA3

2. Project Overview

Click is a Python toolkit for building command-line interfaces. A developer describes commands and options with decorators, and Click handles parsing, validation and the automatic `--help` page. It is developed by the Pallets organization.

Item	Details (seen on the repository page, 12 Oct 2026)
Project	Click, a Python composable command line interface toolkit
Website	click.palletsprojects.com
Language / License	Python / BSD-3-Clause
Topics	cli, click, pallets, python
Popularity	17.8k stars, 4.4k forks, 187 watching
Activity	3,379 commits, 3 branches, 71 tags, 30 releases (latest 8.5.0)
Issues / pull requests	75 open issues (1,735 closed), 9 open pull requests
Last commit seen	"Add Parameter.spec and deprecate human_readable_name (#3877)", commit 2247b35, about a week old

2.1 Why We Chose Click

Criterion	How Click fits
Active maintenance	Recent commits, regular releases and 75 open issues show an active project
Contribution guidance	README, license, code of conduct, contributing and security policy are all linked on the repository page
Reproducible issues	Bugs such as #3362 run on a laptop with a few lines of Python, with no special hardware
Language we know	Python, so we can read and change the code
Syllabus relevance	It exercises issues, bug reproduction, pull requests and the contribution workflow of Unit 3

2.2 Working Demonstration

We cloned Click, created a virtual environment and installed it in editable mode with `pip install -e .`. pip reported `click-8.6.0.dev0`, the development version, which is newer than the latest release (8.5.0). We then wrote a small program with one option and ran it.

```
import click

@click.command()
@click.option("--name", default="World", help="Who to greet")
def hello(name):
    click.echo(f"Hello, {name}!")

if __name__ == "__main__":
    hello()
```

Running `python hello.py --name Pune` printed `Hello, Pune!`, and `python hello.py --help` printed the usage and the `--name` option.

```
C:\Users\bhumi\click>python -m venv venv

C:\Users\bhumi\click>venv\Scripts\activate

(venv) C:\Users\bhumi\click>pip install -e .
Obtaining file:///C:/Users/bhumi/click
  Installing build dependencies ... done
  Checking if build backend supports build_editable ... done
  Getting requirements to build editable ... done
  Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: click
  Building editable for click (pyproject.toml) ... done
  Created wheel for click: filename=click-8.6.0.dev0-py3-none-any.whl size=3144 sha256=396fde03e2318dc5e4b76504cdb6f80c5c30b75263dbb1147f9a8f4750dbeb95
  Stored in directory: C:\Users\bhumi\AppData\Local\Temp\pip-ephem-wheel-cache-5txpfvcj\wheels\ed\f3\e0\4ccd7844f77b354ae59c796a57df71e2b80c41825a6378ac66
Successfully built click
Installing collected packages: click
Successfully installed click-8.6.0.dev0

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(venv) C:\Users\bhumi\click>python hello.py --name Pune
Hello, Pune!

(venv) C:\Users\bhumi\click>python hello.py --name Pune
Hello, Pune!

(venv) C:\Users\bhumi\click>python hello.py --help
Usage: hello.py [OPTIONS]

Options:
  --name TEXT  Who to greet
  --help       Show this message and exit.

(venv) C:\Users\bhumi\click>
```

Figure 1: Click installed from source (8.6.0.dev0) and the demo program running

3. Repository Investigation

We explored the official repository at https://github.com/pallets/click through the browser and through our local clone.

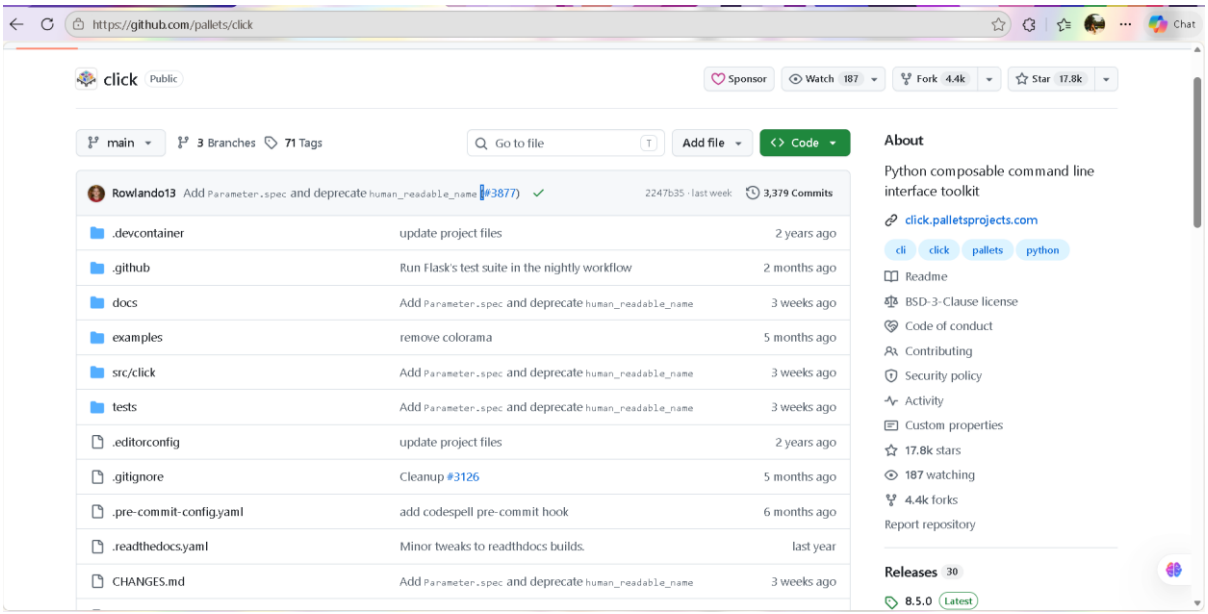


Figure 2: Main page of the pallets/click repository

3.1 Structure

Item	What it holds
`src/click`	The library source code (we read `formatting.py` and `_textwrap.py` here)
`tests`	The automated test suite (pytest), for example `tests/test_formatting.py`

Item	What it holds
`docs`	Documentation source, published at click.palletsprojects.com
`examples`	Small example applications
`.github`	GitHub automation, such as workflows
`.devcontainer`	Development container settings
`pyproject.toml`, `uv.lock`	Project metadata and locked development dependencies
`.pre-commit-config.yaml`	Automatic checks that run before each commit
`CHANGES.md`	Changelog of released and upcoming changes
`README.md`, `LICENSE.txt`	Project description and the BSD-3-Clause license
`.editorconfig`, `.gitignore`, `.readthedocs.yaml`	Editor style, ignored files and documentation build settings

3.2 Project Activity

The most recent commit was made the week before our visit, and the repository has 3,379 commits, 71 tags and 30 releases. Its right-hand panel links the README, license, code of conduct, contributing guide, security policy and an Activity view, so the project documents how to take part.

3.3 Issues, Labels and Pull Requests

The Issues tab shows 75 open and 1,735 closed issues, and the Pull requests tab shows 9 open pull requests. Our first search, `is:issue is:open label:bug`, returned the message **"Invalid value bug for label"**. The reason is that Click spells its labels with capital letters, such as `Bug` and `Feature`. Other labels we saw were `help output`, `parsing`, `typing`, `docs` and `tests`.

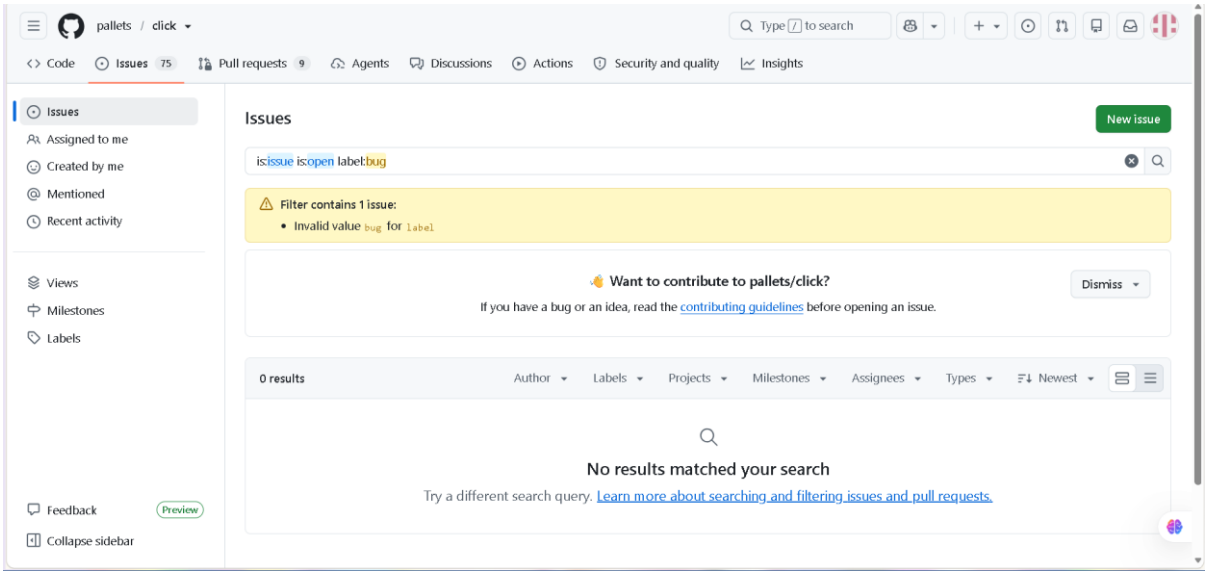


Figure 3: Searching for label:bug gave "Invalid value", because the label is spelt Bug

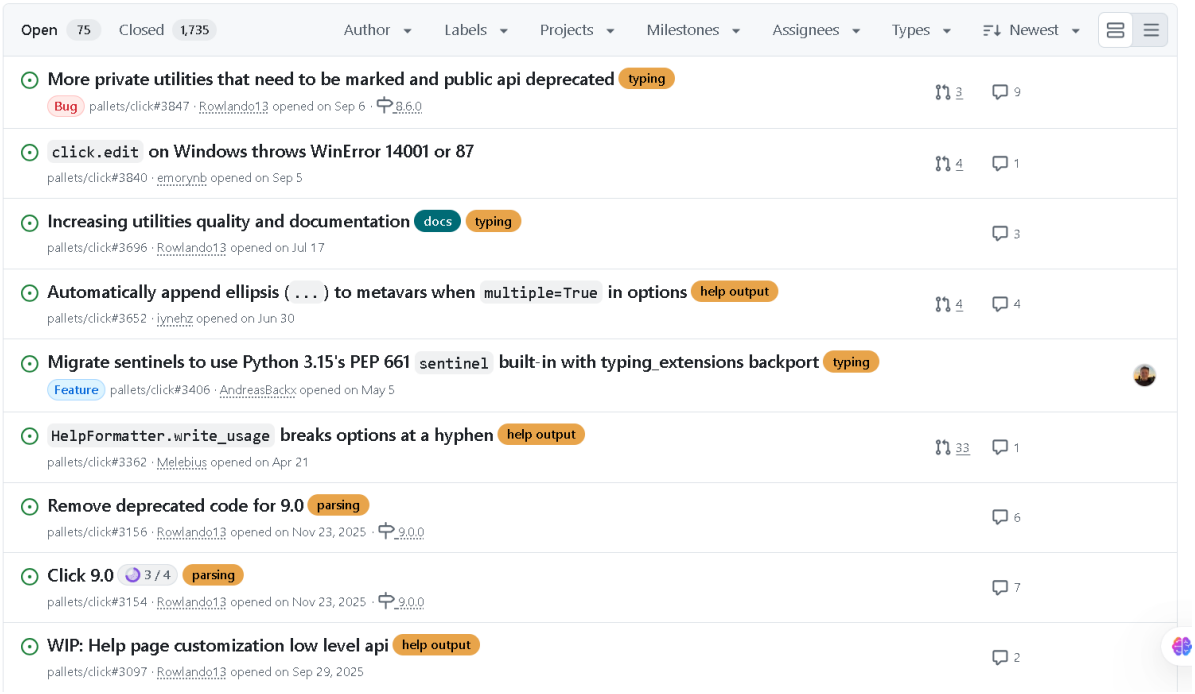


Figure 4: Open issues list (first part), showing labels and linked pull requests

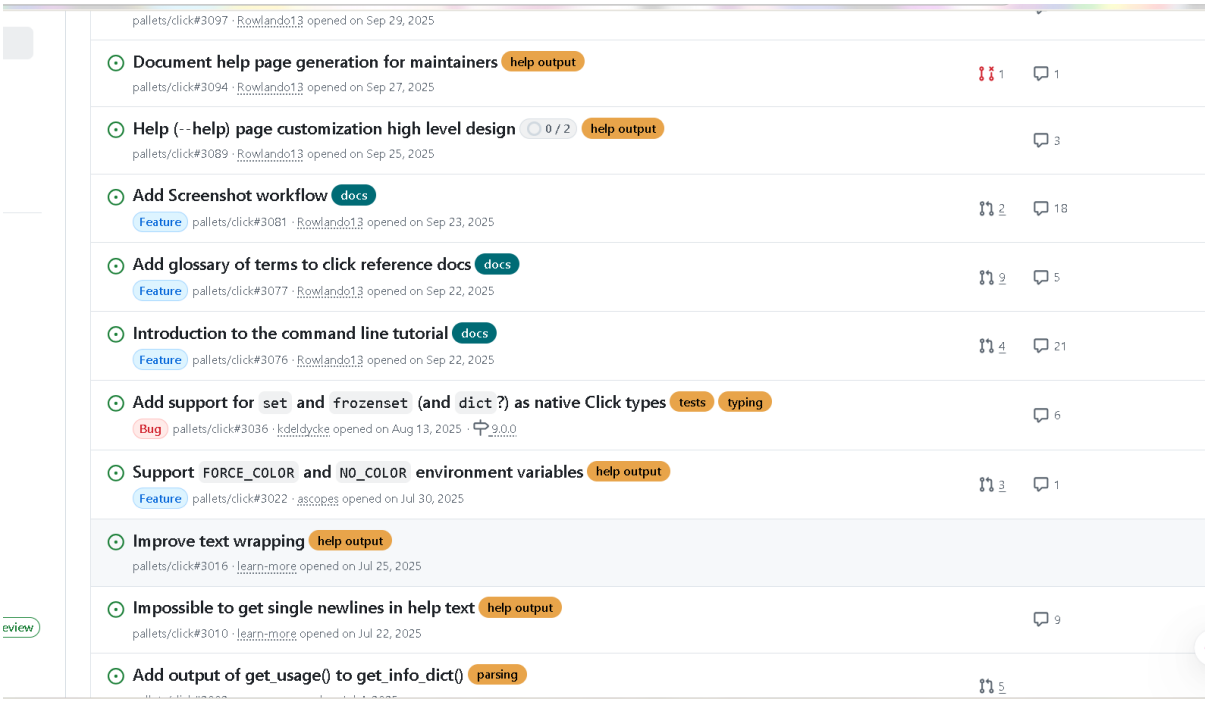


Figure 5: Open issues list (second part)

From this list we chose issue #3362, because it is a real bug with a complete code example, it runs on any operating system, and it has linked pull requests we could study. We avoided maintainer planning issues, issues that were large design discussions, and a Windows-only `click.edit` error that needs an editor set up to reproduce.

4. Issue Investigation

Item	Details
Issue	pallets/click#3362: `HelpFormatter.write_usage` breaks options at a hyphen
Opened by	Melebius, on Apr 21
Label	`help output`
Assignee / milestone	No one assigned, no milestone
Status	Open (at the time of our investigation)
Comments	The issue list shows 1 comment
Linked pull requests	#3385, #3388, #3399 (shown under Development; all three are closed)

Description. The reporter explains that if an option containing a hyphen in its name is printed at the line break limit, it is broken at the hyphen. The report gives a short reproduction, an expected output and the actual output. The reproduction builds a `HelpFormatter` with `width=65` and writes a usage line for ten long options.

Expected result. Each option stays whole, and lines break only between options. **Actual result.** Some options are split at an internal hyphen.

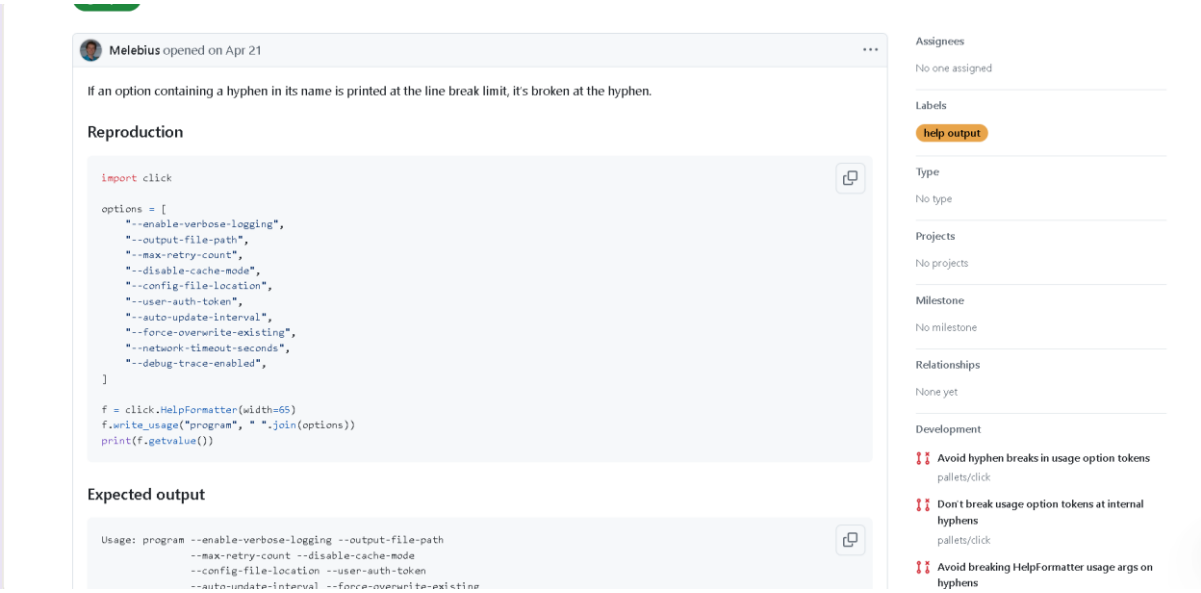


Figure 6: Issue #3362 on GitHub: description, reproduction code, labels and linked pull requests

5. Bug Investigation

5.1 Environment

Item	Details
Operating system	Windows 11 Home, Version 26H2 (OS Build 26300.9550), as shown by winver
Python	3.13.7

Item	Details
Click	8.6.0.dev0, editable install from the cloned `main` branch (commit 2247b35)
Tools	Git, VS Code, PowerShell / Command Prompt, pytest 9.1.1

5.2 Steps to Reproduce

1. Clone the repository: `git clone https://github.com/pallets/click.git`, then `cd click`.
2. Create and activate a virtual environment: `python -m venv venv`, then `venv\Scripts\activate`.
3. Install Click in editable mode: `pip install -e .`
4. Create `reproduce_3362.py` with the code from the issue (below).
5. Run `python reproduce_3362.py`.

```
import click

options = [
    "--enable-verbose-logging", "--output-file-path", "--max-retry-count",
    "--disable-cache-mode", "--config-file-location", "--user-auth-token",
    "--auto-update-interval", "--force-overwrite-existing",
    "--network-timeout-seconds", "--debug-trace-enabled",
]

f = click.HelpFormatter(width=65)
f.write_usage("program", " ".join(options))
print(f.getvalue())
```

5.3 Expected and Actual Result

	Result
Expected	Usage wraps only between whole options; no option is split at a hyphen
Actual	Three options were split: `--max-` / `retry-count`, `--config-file-` / `location`, and `--network-timeout-` / `seconds`
Status	**Reproduced** on the latest development version (8.6.0.dev0), so the bug is still present

```
Usage: program --enable-verbose-logging --output-file-path --max-
               retry-count --disable-cache-mode --config-file-
               location --user-auth-token --auto-update-interval
               --force-overwrite-existing --network-timeout-
               seconds --debug-trace-enabled
```

```
Options:
  --name TEXT  Who to greet
  --help       Show this message and exit.

(venv) C:\Users\bhumi\click>python reproduce_3362.py
Usage: program --enable-verbose-logging --output-file-path --max-
               retry-count --disable-cache-mode --config-file-
               location --user-auth-token --auto-update-interval
               --force-overwrite-existing --network-timeout-
               seconds --debug-trace-enabled


(venv) C:\Users\bhumi\click>_
```

Figure 7: Our reproduction output: three options are split across lines

```
(venv) C:\Users\bhumi\click>python experiment_3362.py
Default (break_on_hyphens=True):
--enable-verbose-logging --output-file-
path --max-retry-count

With break_on_hyphens=False:
--enable-verbose-logging
--output-file-path --max-retry-count

(venv) C:\Users\bhumi\click>python --version
Python 3.13.7

(venv) C:\Users\bhumi\click>pip show click
Name: click
Version: 8.6.0.dev0
Summary: Composable command line interface toolkit
Home-page:
Author:
Author-email:
License-Expression: BSD-3-Clause
Location: C:\Users\bhumi\click\venv\Lib\site-packages
Editable project location: C:\Users\bhumi\click
Requires:
Required-by:

(venv) C:\Users\bhumi\click>python reproduce_3362.py > output_3362.txt

(venv) C:\Users\bhumi\click>type output_3362.txt
Usage: program --enable-verbose-logging --output-file-path --max-
               retry-count --disable-cache-mode --config-file-
               location --user-auth-token --auto-update-interval
               --force-overwrite-existing --network-timeout-
               seconds --debug-trace-enabled


(venv) C:\Users\bhumi\click>winver
```

Figure 8: Environment evidence: Python 3.13.7, Click 8.6.0.dev0 (editable), the experiment and the saved output

5.4 Finding the Root Cause

We followed the code from the method that prints the usage line.

1. **`HelpFormatter.write_usage`** (`src/click/formatting.py`) receives the arguments as one whitespace-separated string. In both of its layouts (arguments beside the prefix, or on the next line when the prefix is long) it hands that string to `wrap_text`.
2. **`wrap_text`** (same file) creates a `TextWrapper` with only `width`, `initial_indent`, `subsequent_indent` and `replace_whitespace=False`. It never sets `break_on_hyphens`.
3. **`TextWrapper`** (`src/click/_textwrap.py`) is declared as `class TextWrapper(textwrap.TextWrapper)`, so it inherits Python's standard behaviour. A search for `break_on_hyphens` in that file found no results.

```python
def write_usage(self, prog: str, args: str = "", prefix: str | None = None) -> None:
    """Writes a usage line into the buffer.

    :param prog: the program name.
    :param args: whitespace separated list of arguments.
    :param prefix: The prefix for the first line. Defaults to
        ``"Usage: "``.
    """
    if prefix is None:
        prefix = "{usage} ".format(usage=_("Usage:"))

    usage_prefix = f"{prefix:>{self.current_indent}}{prog} "
    text_width = self.width - self.current_indent

    if not args:
        # Without args, the prefix's trailing space and the wrap_text
        # call that would normally place args on the line are both
        # unnecessary. Emit just the prefix line.
        self.write(usage_prefix.rstrip(" "))
        self.write("\n")
        return

    if text_width >= (term_len(usage_prefix) + 20):
        # The arguments will fit to the right of the prefix.
        indent = " " * term_len(usage_prefix)
        self.write(
```

Figure 9: write_usage in formatting.py (top part)

```python
179
180     if text_width >= (term_len(usage_prefix) + 20):
181         # The arguments will fit to the right of the prefix.
182         indent = " " * term_len(usage_prefix)
183         self.write(
184             wrap_text(
185                 args,
186                 text_width,
187                 initial_indent=usage_prefix,
188                 subsequent_indent=indent,
189             )
190         )
191     else:
192         # The prefix is too long, put the arguments on the next line.
193         self.write(usage_prefix)
194         self.write("\n")
195         indent = " " * (max(self.current_indent, term_len(prefix)) + 4)
196         self.write(
197             wrap_text(
198                 args, text_width, initial_indent=indent, subsequent_indent=indent
199             )
200         )
201
202     self.write("\n")
203
204 def write_heading(self, heading: str) -> None:
205     """Writes a heading into the buffer."""
```

Figure 10: write_usage in formatting.py: both branches call wrap_text(args, ...)

```
58          ``text``, ``initial_indent``, or ``subsequent_indent`` no longer
59          count toward the width budget, so styled input wraps based on what
60          the user sees instead of raw byte length.
61      """
62      from ._textwrap import TextWrapper
63
64      text = text.expandtabs()
65      wrapper = TextWrapper(
66          width,
67          initial_indent=initial_indent,
68          subsequent_indent=subsequent_indent,
69          replace_whitespace=False,
70      )
71      if not preserve_paragraphs:
72          return wrapper.fill(text)
73
74      p: list[tuple[int, bool, str]] = []
75      buf: list[str] = []
76      indent = None
77
78      def _flush_par() -> None:
79          if not buf:
80              return
81          if buf[0].strip() == "\b":
82              p.append((indent or 0, True, "\n".join(buf[1:])))
83          else:
84              p.append((indent or 0, False, " ".join(buf)))
```

Figure 11: wrap_text creates TextWrapper(...) without break_on_hyphens

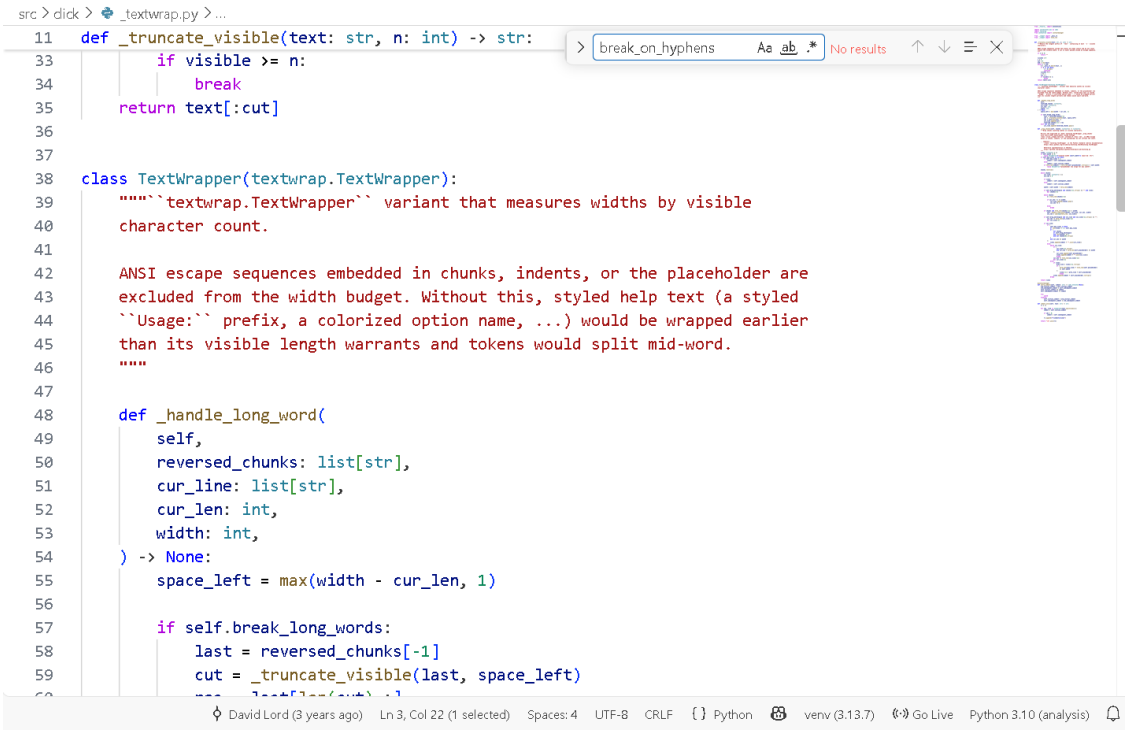


Figure 12: _textwrap.py: TextWrapper subclasses textwrap.TextWrapper; searching break_on_hyphens gives "No results"

Because `args` is a single string, `write_usage` cannot tell where one option ends. Python's `textwrap` treats a hyphen inside a word as a place where a line may break (`break_on_hyphens` defaults to `True`). That is helpful for ordinary prose but wrong for option names, so a long option that reaches the line limit is split at one of its hyphens.

5.5 Supporting Experiment

To check this explanation, we wrapped three option names with Python's own `textwrap`, once with the default and once with `break_on_hyphens=False`.

```
import textwrap
text = "--enable-verbose-logging --output-file-path --max-retry-count"
print(textwrap.fill(text, width=40))
print(textwrap.fill(text, width=40, break_on_hyphens=False))
```

Setting	Output
Default (`break_on_hyphens=True`)	`--enable-verbose-logging --output-file-` / `path --max-retry-count` (option split)
`break_on_hyphens=False`	`--enable-verbose-logging` / `--output-file-path --max-retry-count` (options whole)

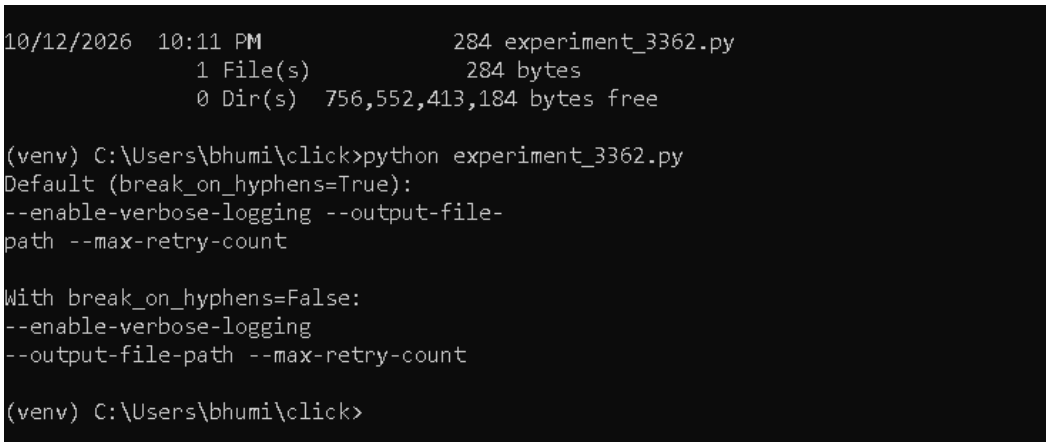


Figure 13: Experiment output: default wrapping splits --output-file-path; with break_on_hyphens=False it stays whole

The experiment shows the mechanism. Together with the code reading above, it explains the bug: the default hyphen breaking is never switched off on the usage path. We write this as a finding from reading the code and running the experiment, not as the maintainers' statement.

6. Contribution Process

Click is part of the Pallets projects, so contributions follow the shared Pallets guide at palletsprojects.com/contributing. The pages below summarise what that guide says, in our own words.

6.1 What to Work On

Anyone may work on any open ticket without asking first. Before starting, check whether someone is assigned to the issue or whether a pull request is already linked. Besides code, helpful contributions include better documentation, better tests, type annotations and answering questions.

6.2 Workflow

1. **Fork and clone** the project (the guide uses the GitHub CLI: `gh repo fork --clone pallets/<project>`).
2. **Choose the target branch.** Bug and documentation fixes go to the `stable` branch if the project has one; otherwise they go to `main`. Pull from `upstream` to update it. (Click shows 3 branches; we did not check which of them is `stable`.)
3. **Create a work branch** with a short descriptive name, for example `git switch -c short-descriptive-name`.

4. **Set up the environment.** Create a virtual environment and install the dependencies in editable mode. Projects with a `requirements` directory use `pip install -r requirements/dev.txt` and `pip install -e .`; projects without it use `uv sync`. Then run `pre-commit install --install-hooks`. On Windows the guide asks for PowerShell, not CMD. Click's folder contains `uv.lock`; we installed with `pip install -e .`, which was enough to run and test the code.
5. **Make the change and add tests** that show it works, change the documentation if needed, and make sure all tests pass. A changelog entry is optional, and a maintainer will write one if needed. Documentation-only changes need no entry.
6. **Open the pull request** against the target branch (`gh pr create --web --base stable`). CI runs automatically. Fix any failures by pushing new commits, then click **Ready for review**.
7. **Review.** A maintainer reviews and merges when available.

6.3 Checks Used While Developing

Command	Purpose
`pytest`	Run the unit tests
`mypy`	Run the main type checker
`pre-commit`	Run the linter and formatter on staged files; it also runs before each commit
`tox run -e docs,docs-auto`	Build the documentation and preview it locally
`tox parallel`	Run all test environments that CI runs

6.4 LLM and AI Policy

Every contribution must also follow the Pallets Code of Conduct and the **LLM and AI Policy**. In short: contributions must be understood and created by humans; contributions that appear generated by LLM or AI tools will be closed and the author is likely to be blocked; new contributors are told not to use such tools at all. The policy covers code, tests, pull request descriptions and comments. This policy matters for our study, because the three pull requests for issue #3362 all carry the label "rejected AI" (Section 7).

7. Pull Request Investigation

Issue #3362 lists three linked pull requests under Development. We opened each one and recorded what it says.

PR	Title	Author	Opened	Status
#3385	Avoid hyphen breaks in usage option tokens	cyphercodes	Apr 27	Closed, not merged
#3388	Don't break usage option tokens at internal hyphens	charlieleith	Apr 29	Closed, not merged
#3399	Avoid breaking HelpFormatter usage args on hyphens	MackDing	May 2	Closed, not merged

7.1 PR #3385: Avoid hyphen breaks in usage option tokens

Opened by cyphercodes from the branch `fix-usage-hyphen-wrap-3362` into `pallets:main`, with 1 commit ("fix: avoid hyphen breaks in usage options", 0b5fdd7), 11 checks and 3 changed files. The description says it prevents `write_usage` from wrapping hyphenated option tokens at internal hyphens when the full token can fit on the next line, and that it adds a regression test and a changelog entry. It lists the test commands it ran. The maintainer **davidism closed it on the day it was opened (Apr 27)**, and the github-actions bot later locked the conversation (May 12). It has no reviews, no comments and the label **rejected AI**.

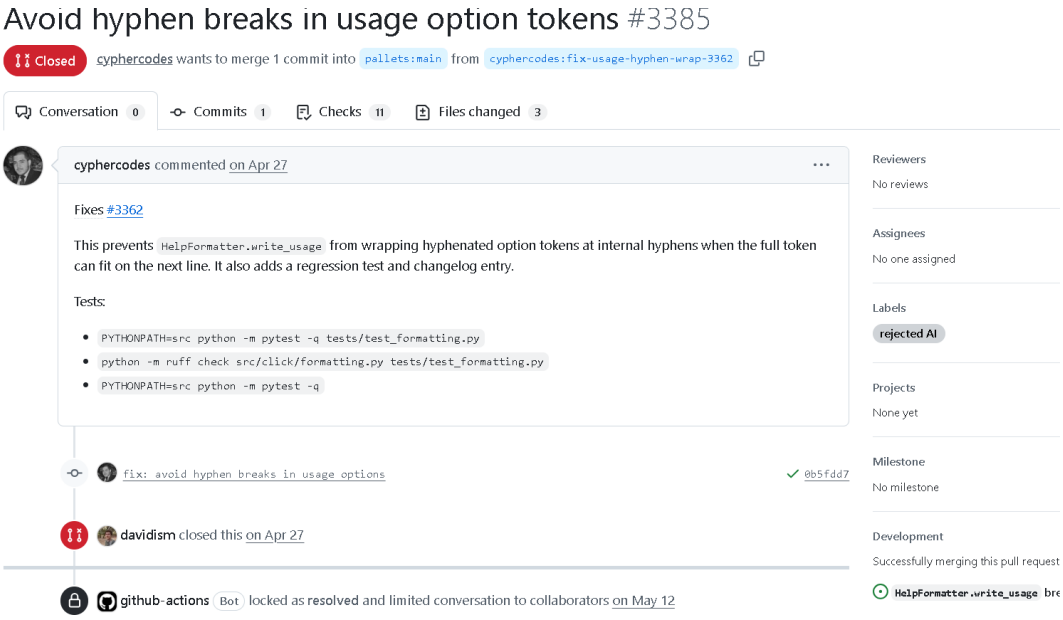


Figure 14: PR #3385: closed, no reviews, label "rejected AI"

7.2 PR #3388: Don't break usage option tokens at internal hyphens

Opened by charlieleith on Apr 29 from the branch `fix-3362-usage-hyphen-wrap`, with 1 commit. Its description states the same cause we found: `write_usage` passes its argument string through `wrap_text`, which builds a `TextWrapper` whose `break_on_hyphens` defaults to `True`. It calls this the right choice for prose but the wrong one for usage text, where each token is a long flag. It has no reviews, is closed, and carries the label **rejected AI**.

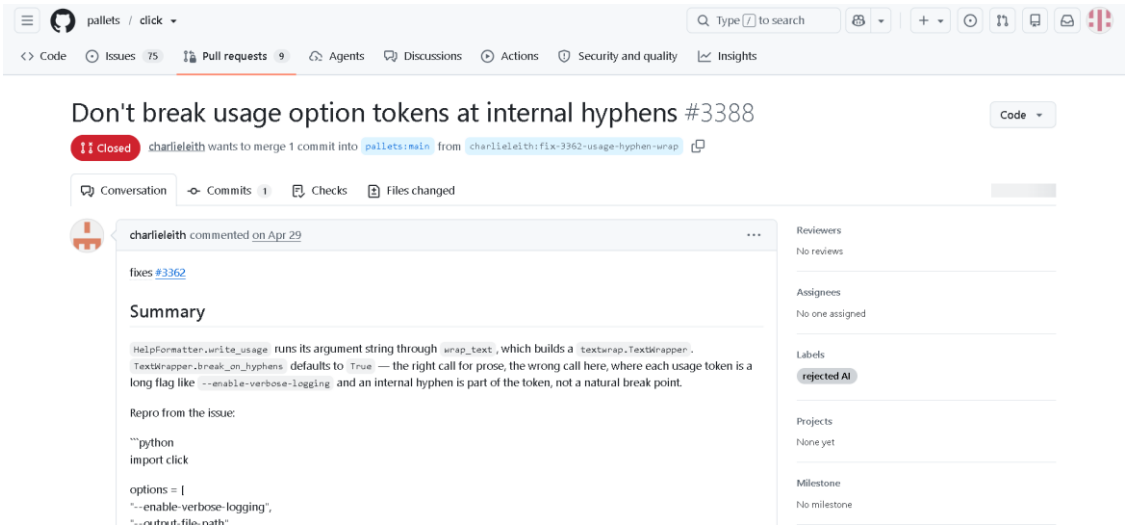


Figure 15: PR #3388: closed, no reviews, label "rejected AI"

7.3 PR #3399: Avoid breaking HelpFormatter usage args on hyphens

Opened by MackDing on May 2 from the branch `fix-helpformatter-usage-hyphen-break-3362`, with 1 commit. Its summary says it prevents `write_usage` from breaking inside hyphenated option names, keeps the existing wrapping for other `wrap_text` callers by adding explicit wrapper controls, and adds a regression test for the reported example. It also quotes the same "before" output as the issue. It has no reviews, is closed, and carries the label **rejected AI**.

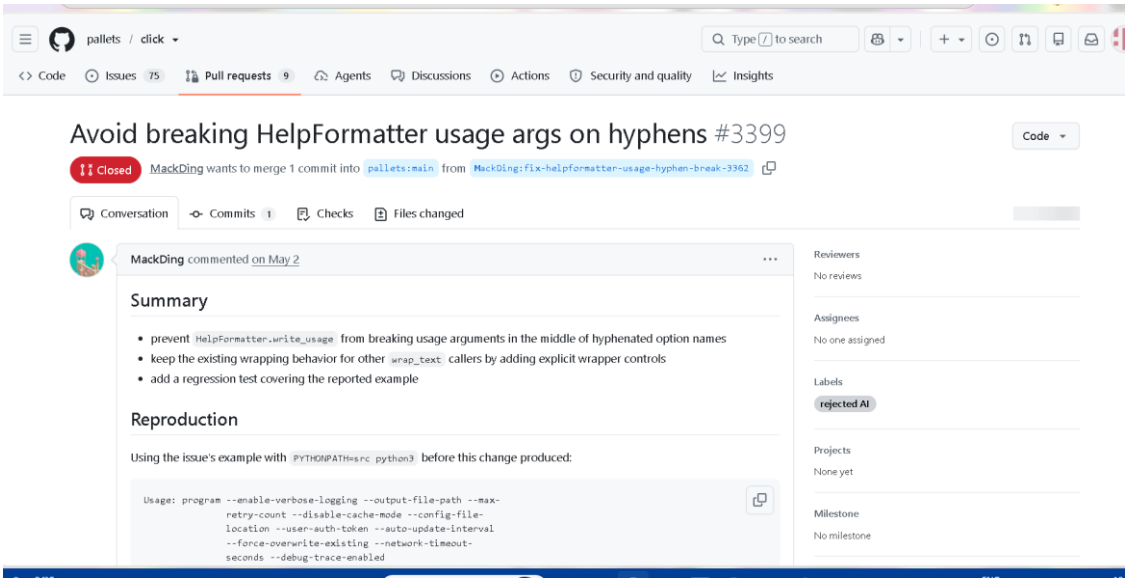


Figure 16: PR #3399: closed, no reviews, label "rejected AI"

7.4 Comparison and Observations

Point	#3385	#3388	#3399
Cause identified	Hyphen wrapping in `write_usage`	`break_on_hyphens=True` default	Hyphen wrapping in `write_usage`
Approach (from description)	Avoid the break when the whole token fits on the next line	Treat usage tokens differently from prose	Explicit wrapper controls so other callers keep old behaviour
Test / changelog	Test and changelog entry	Not recorded	Regression test
Reviews	None	None	None
Outcome	Closed same day, locked later	Closed	Closed

- All three PRs agree on the root cause, which matches our own finding.
- None was merged and none received a technical review, so there is no maintainer feedback on which approach is preferred.
- All three carry the label "rejected AI". This fits the Pallets LLM and AI policy in Section 6.4. It is our inference from the label and the policy; the pull requests do not give a longer explanation.
- PR #3399's idea of limiting the change to usage text is important, because `wrap_text` is also used for ordinary help text (see Section 9.4).

8. Practical Observations

These are the problems we met while doing the work and what they taught us.

Problem	What happened and how we solved it
Label search failed	`label:bug` returned "Invalid value". Click's labels are capitalised (`Bug`), so we used the label list and clicked the label instead.
Wrong folder	We ran `pip install -e .` in the home folder and pip said it was not a Python project. We cloned Click, ran `cd click` so the prompt ended in `\click>`, and installed again.
Commands on one line	Two commands typed on one line gave "The system cannot find the path". We ran one command per line.
File saved in the wrong place	Our experiment script was saved in `src\click`, so `python experiment_3362.py` said "No such file". Git showed it as `src/click/experiment_3362.py`. We moved it to the project root.
`types.py` shadowing	Running a file from inside `src\click` caused `ImportError: cannot import name 'WrapperDescriptorType' from 'types'`. Click has its own `types.py`, which hid Python's standard `types` module. The lesson: run our own scripts from the project root and never run library source files directly.
Git push rejected	Pushing to our case-study repository failed with "fetch first", because we had edited files on the GitHub website. We ran `git pull --rebase` and pushed again.
Untracked file marker	VS Code showed a "U" next to new files. It means Git does not track the file yet; it is not an error.
Messy diff	Our first `git diff` showed whole lines removed and re-added because we had lost the 4-space indentation. We re-indented the lines so the final change is a single added line.

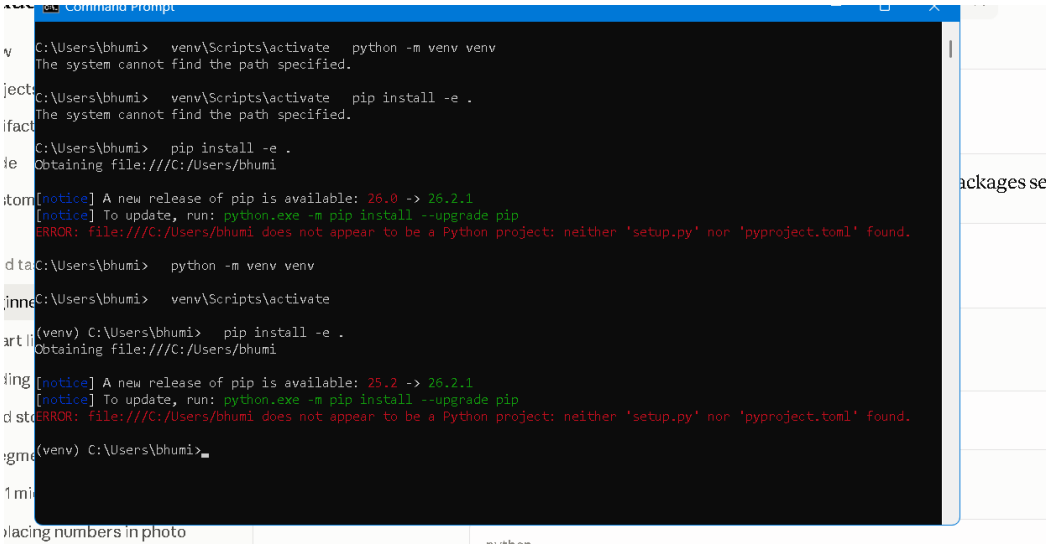


Figure 17: pip install -e . run in the wrong folder (home folder instead of click)

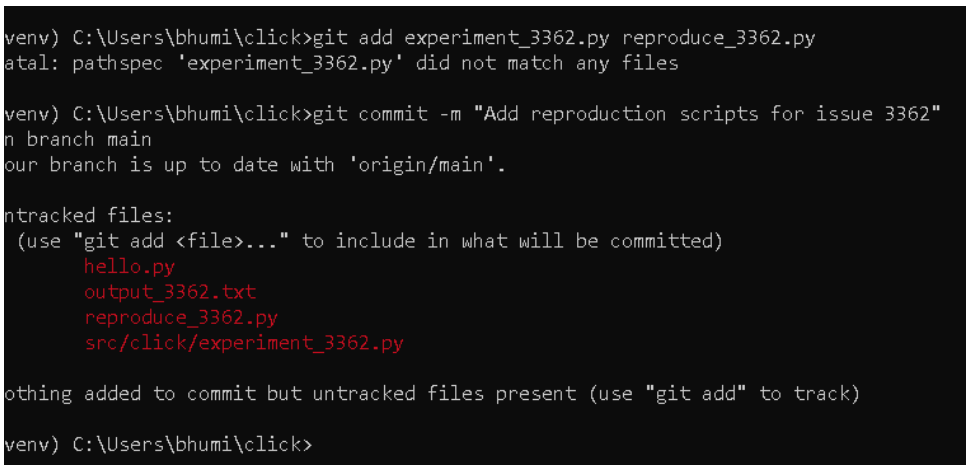


Figure 18: git status listing src/click/experiment_3362.py: the file was saved in the wrong folder

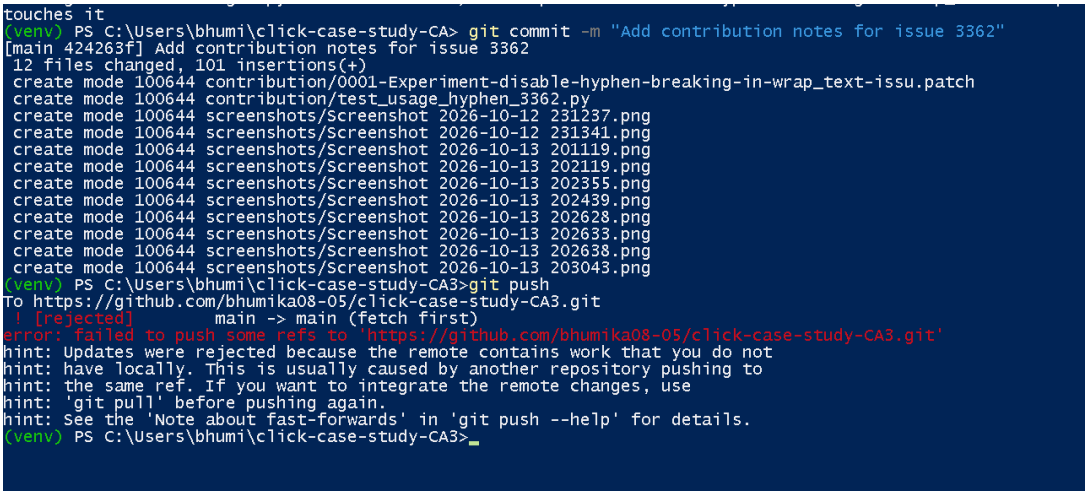


Figure 19: git push rejected ("fetch first"), fixed with git pull --rebase

What we learned

- The bug is **still present on the development version**, even though three pull requests were opened for it. An old issue is not necessarily fixed.
- A real project has its own conventions (label spelling, branches, policies), and reading them first saves time.
- Reproducing a bug with the reporter's exact code is quick, and an extra small experiment helps explain the cause.
- The Pallets AI policy shows that open source projects value understanding and trust, not only working code.
- Keeping the work in a public repository with small commits makes it easy to show progress.

9. Changes / Contribution

Our contribution has two parts: an automated test that detects the bug, and a one-line experimental change that makes the test pass. Both were made in our local clone and our own repository.

9.1 The Test

We wrote `test_usage_hyphen_3362.py`. It builds the usage line from the issue (ten options, `HelpFormatter(width=65)`) and has two tests. The first checks that every option still appears whole in the output; the second splits the output on whitespace and checks that the pieces equal the original options.

```
import click

OPTIONS = [
    "--enable-verbose-logging", "--output-file-path", "--max-retry-count",
    "--disable-cache-mode", "--config-file-location", "--user-auth-token",
    "--auto-update-interval", "--force-overwrite-existing",
    "--network-timeout-seconds", "--debug-trace-enabled",
]

def make_usage():
    formatter = click.HelpFormatter(width=65)
    formatter.write_usage("program", " ".join(OPTIONS))
    return formatter.getvalue()

def test_options_are_not_split_at_hyphens():
    usage = make_usage()
    for option in OPTIONS:
        assert option in usage, f"{option} was split across lines"

def test_usage_tokens_match_options():
    tokens = make_usage().split()[2:]  # skip "Usage:" and "program"
    assert tokens == OPTIONS
```

9.2 Result Before the Change

Running `pytest test_usage_hyphen_3362.py -v` on the unchanged code gave **2 failed**. The messages were `--max-retry-count was split across lines` and `'--max-' != '--max-retry-count'`, with three extra pieces such as `'--network-timeout-'`. This proves that the bug exists in the current code.

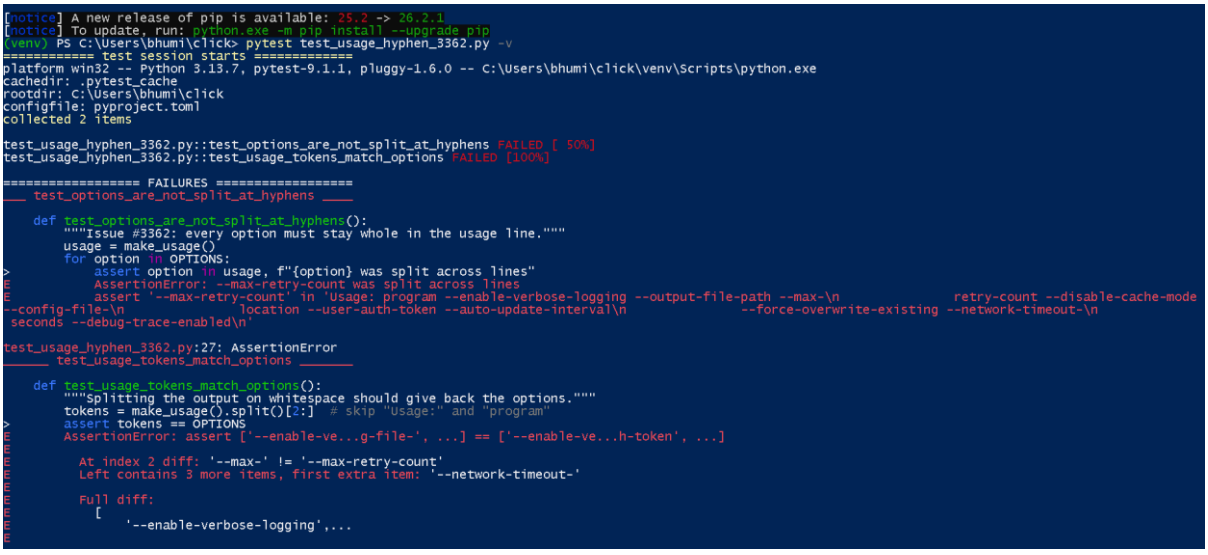


Figure 20: Both tests FAILED on the unchanged code

9.3 The Experimental Change

In `wrap_text` (`src/click/formatting.py`), we added one argument to the `TextWrapper(...)` call:

```
wrapper = TextWrapper(
    width,
    initial_indent=initial_indent,
    subsequent_indent=subsequent_indent,
    replace_whitespace=False,
+   break_on_hyphens=False,
)
```

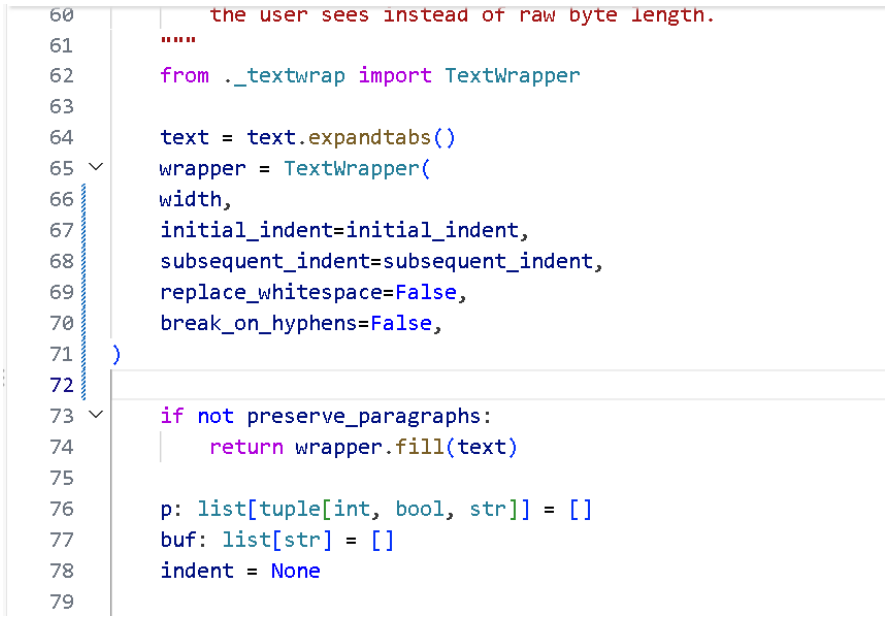


Figure 21: The changed TextWrapper call in wrap_text (line 70)

9.4 Results After the Change

Check	Result
Our two tests	2 passed in 0.04 s (were 2 failed)
`tests/test_formatting.py`	37 passed
Our tests and `test_formatting.py` together	39 passed in 0.16 s
Whole Click test suite (`pytest -q`)	2178 passed, 108 skipped, 1 xfailed, 31000 deselected, none failed (9.63 s)
Git commit	`8ada84d` on branch `experiment-fix-3362`: 2 files changed, 35 insertions(+)

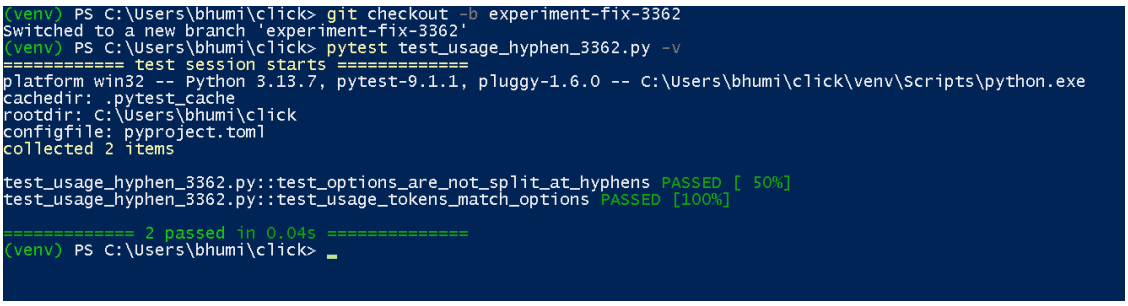


Figure 22: Both tests PASSED after the change (branch experiment-fix-3362)

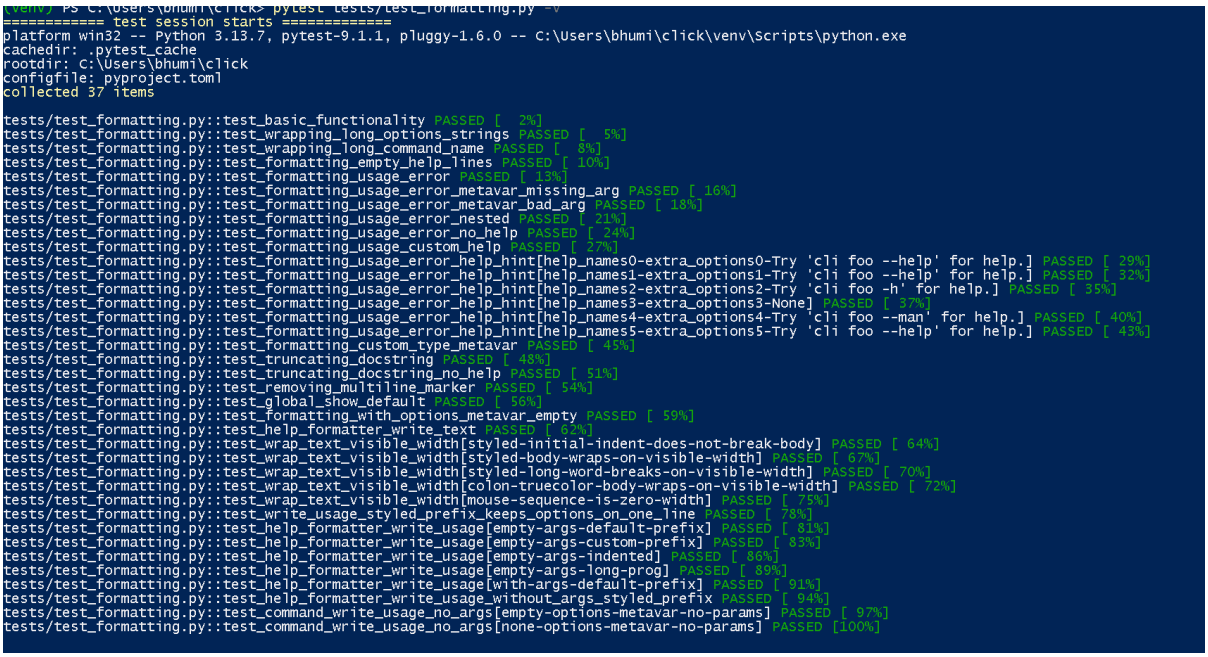


Figure 23: All 37 existing formatting tests still pass

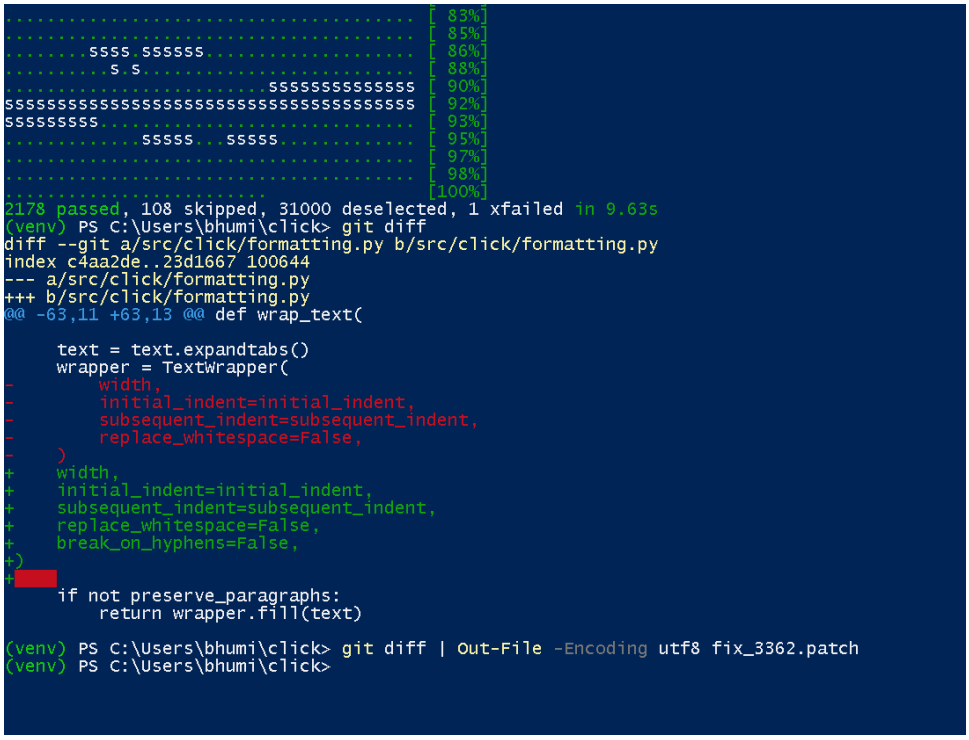


Figure 24: Full test suite summary (2178 passed) and git diff of the change

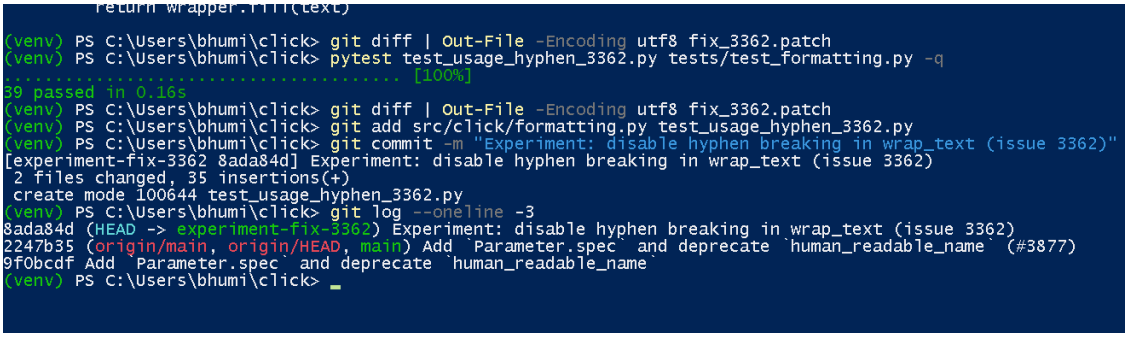


Figure 25: Commit 8ada84d on top of the official main commit 2247b35

9.5 Limits of This Change

- `wrap_text` is used for **all** wrapped help text, not only the usage line. Setting `break_on_hyphens=False` there stops hyphen breaking everywhere it is used, so this is an experiment, not a production-ready fix.
- The existing tests passing does not prove that ordinary help text wraps as before, because those tests may not cover hyphenated words in descriptions.
- A better fix would limit the change to the usage line, which is what PR #3399 tried to do with explicit wrapper controls.
- Our work is a **local experiment and a test in our own repository**. We did **not** submit it to the official Click repository, because the Pallets LLM and AI policy asks new contributors to create contributions themselves and without AI tools, and the three earlier pull requests for this issue were closed with the label "rejected AI".

9.6 GitHub Work

All evidence is kept in the public repository https://github.com/bhumika08-05/click-case-study-CA3. It contains a README describing the case study, a `reproduction` folder (scripts and output), a `screenshots` folder, a `notes` folder (root cause, linked pull requests and contribution notes) and a `contribution` folder (our test and the patch of our change). We pushed in small steps with clear commit messages; the repository showed 8 commits at the time of the screenshot below, with the latest commit "Add contribution notes for issue 3362".

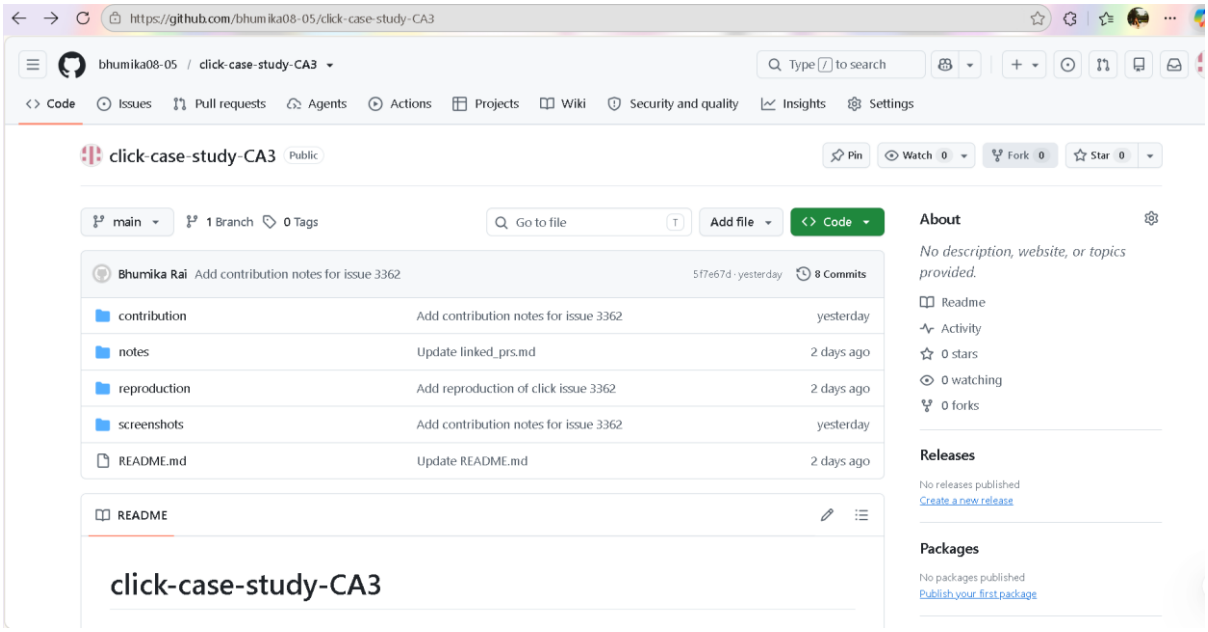


Figure 26: Our case-study repository on GitHub with the contribution, notes, reproduction and screenshots folders (8 commits)

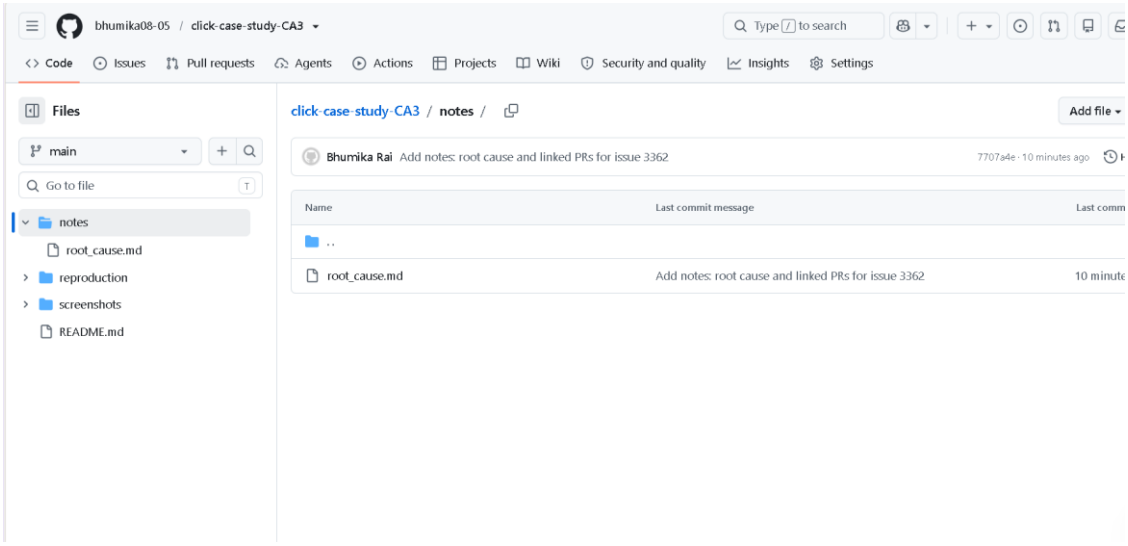


Figure 27: The notes folder with root_cause.md in our repository

10. Conclusion

We investigated pallets/click#3362 from start to finish. We reproduced the bug on Click 8.6.0.dev0 on Windows with Python 3.13.7 and found that three options are split at hyphens. By reading `write_usage`, `wrap_text` and `_textwrap.py`, we found that Click's `TextWrapper` inherits Python's default `break_on_hyphens=True` and never turns it off on the usage path, and a small experiment with Python's `textwrap` supported this explanation. We studied the Pallets contribution process, including its LLM and AI policy, and examined three earlier pull requests that identified the same cause but were closed unmerged with the label "rejected AI". Finally, we wrote a test that fails on the current code and passes with a one-line experimental change, checked that the existing tests still pass, and noted that a safe fix must be limited to the usage line. The main lesson is that contributing is as much about understanding the project's process and being honest about results as it is about writing code.

References

1. Click official repository: https://github.com/pallets/click
2. Click documentation: https://click.palletsprojects.com
3. Issue #3362: https://github.com/pallets/click/issues/3362
4. Pull request #3385: https://github.com/pallets/click/pull/3385
5. Pull request #3388: https://github.com/pallets/click/pull/3388
6. Pull request #3399: https://github.com/pallets/click/pull/3399
7. Pallets, Contribute to a Pallets Project: https://palletsprojects.com/contributing/
8. Pallets, Contributing Quick Reference: https://palletsprojects.com/contributing/quick
9. Pallets, LLM and AI Policy: https://palletsprojects.com/contributing/llm-ai
10. Python documentation, textwrap: https://docs.python.org/3/library/textwrap.html
11. Our case-study repository: https://github.com/bhumika08-05/click-case-study-CA3